

Agile & DevOps per la Tua Azienda: Senza Certificazioni Inutili e S

Guida pratica per manager, CTO e CEO di PMI italiane

di Elios Scoglio — 108 Vision

Era un martedì sera, ore 21:30. Il team era ancora in call. Guardavo una board Jira con 47 ticket "In Progress" aperti da tre settimane. Nessuno era davvero in avanzamento: tutti bloccati su un'approvazione, una dipendenza irrisolta, una specifica che nessuno aveva scritto. Avevamo fatto Agile alla lettera — cerimonie, board, punti storia, standup quotidiani — e il software non usciva. Il team era esausto. I clienti aspettavano.

Quello che ho imparato dopo oltre 20 anni su sistemi critici — gestendo team fino a 20 persone, portando aziende da zero a delivery funzionante — è che la differenza tra Agile che funziona e Agile che è teatro si misura in tre cose: chiarezza sullo scopo, disciplina nelle basi, e coraggio di cambiare quello che non funziona invece di continuare a fare le cerimonie.

Questo manuale è il distillato di quel percorso. In queste pagine troverai metodi applicabili da subito, i numeri reali di trasformazioni riuscite, e un piano operativo per i tuoi prossimi 90 giorni.

1. I Numeri che Contano

Prima della teoria, i risultati. Queste sono metriche misurate su team reali, sistemi in produzione, con pressioni di business vere.

Azienda	Metrica	Prima	Dopo
Aruba	Bug rate in produzione	baseline	-98% in 4 mesi
Aruba	Frequenza di deployment	baseline	+400%
Toscana Immobiliare	Lead time to deploy	baseline	-60%
TicketOne	Velocità di sviluppo	baseline	+30%
TicketOne	Soddisfazione del team	baseline	+50%

Questi numeri non vengono da un libro. Vengono da aziende con sistemi in produzione, scadenze reali, e nessuno spazio per esperimenti accademici. Alcuni di questi sistemi gestivano vendita di biglietti per eventi sportivi con obbligo di compliance verso la Polizia di Stato. Non c'era spazio per il fallimento.

Non ho una certificazione Agile da mostrare. Ho i numeri.

***Insight 108 Vision** — Un team che deploya quattro volte più spesso di prima non ha acquistato un tool migliore. Ha cambiato il modo in cui pensa alla consegna del valore. La tecnologia segue la mentalità, non il contrario.*

2. Cos'è Agile Davvero (e Cosa Non E')

Nel 2001, diciassette sviluppatori software scrissero un documento di mezza pagina che cambiò il modo in cui il software viene costruito. Il Manifesto Agile. Quattro valori, dodici principi. In vent'anni, il mondo delle aziende li ha trasformati in qualcosa di irriconoscibile.

Ecco i quattro valori tradotti in italiano operativo per il 2026.

Individui e interazioni sopra processi e strumenti. Due persone che si parlano per 15 minuti risolvono più problemi di cinque ticket su Jira lasciati aperti per una settimana. Quando il processo entra in conflitto con una conversazione utile, vince la conversazione. Se nel tuo team tutto passa per ticket, approvazioni formali e catene di email, hai un problema di collaborazione che nessun tool risolve.

Software funzionante sopra documentazione esaustiva. L'obiettivo è consegnare qualcosa che funziona. La documentazione serve quando serve: per decisioni architetturali rilevanti, per API pubbliche, per compliance. Non come rito burocratico per dimostrare che si è "fatto correttamente". Il software funzionante è la prova che hai fatto il tuo lavoro.

Collaborazione col cliente sopra negoziazione dei contratti. Mostra qualcosa di funzionante ogni due settimane, raccogli feedback, aggiusta la rotta. Il cliente non aspetta sei mesi per scoprire che il prodotto non va bene. Questo funziona solo se il cliente è disponibile a collaborare durante il processo — è un cambiamento culturale anche per lui.

Rispondere al cambiamento sopra seguire un piano. Il piano è uno strumento, non un contratto con l'universo. I team che consegnano puntualmente il prodotto sbagliato non stanno pianificando — stanno recitando. Pianifica in orizzonte breve con alta confidenza, in orizzonte lungo con alta flessibilità.

Le tre bugie che ti hanno raccontato

"Agile significa fare standup ogni mattina." Lo standup mattutino è la cerimonia più abusata. In molte aziende è diventato un rito di aggiornamento di stato dove ciascuno dice cosa ha fatto ieri, con tutti gli altri che aspettano il proprio turno. Non è uno standup Agile — è una riunione di status in piedi. Il Daily Standup serve per sincronizzare il team e far emergere blocchi, non per aggiornare il manager.

"Agile significa niente documentazione." È la scusa più usata per non documentare nulla. Il Manifesto dice "sopra", non "invece di". La documentazione esaustiva non serve. La documentazione essenziale — Architecture Decision Record, contratti API, README utili — è fondamentale. Un team che non documenta le decisioni architetturali accumula debito cognitivo. Sei mesi dopo, nessuno ricorda perché una scelta è stata fatta.

"Agile significa niente pianificazione." Esattamente il contrario. Agile richiede pianificazione continua e di alta qualità su orizzonti brevi. Pianifici due settimane, esegui, misuri, pianifici di nuovo. E più disciplinato del waterfall, non meno.

***Insight 108 Vision** — La distinzione che conta di più non è tra Scrum e Kanban. È tra Agile come processo (cerimonie da seguire) e Agile come mentalità (un modo di prendere decisioni). Puoi seguire tutte le cerimonie alla lettera e avere un team disfunzionale. Puoi non avere nessuna cerimonia formale e fare Agile eccellente.*

3. Perché le Trasformazioni Agile Falliscono in Italia

Ho visto queste dinamiche molte volte. I pattern si ripetono.

Pattern 1: si agilitizza il software ma non cambia niente intorno. Un team fa sprint, tiene gli standup, usa Jira. Ma il CEO continua a chiamare direttamente gli sviluppatori con richieste urgenti che saltano il backlog. Le priorità cambiano ogni settimana. Il Product Owner non ha autorità per dire no. Ogni sprint si conclude con il 40% delle storie spostate al successivo. Il problema non era tecnico — era di governance.

Pattern 2: il project manager diventa Scrum Master senza cambiare ruolo. Lo Scrum Master continua ad assegnare task, decidere priorità, gestire dipendenze in modo centralizzato. Il team non si auto-organizza mai. La retrospective è gestita come check-in di stato. Scrum Master non è project manager con un nome diverso.

Pattern 3: si fa Scrum ma si gestisce come waterfall. Le storie vengono scritte all'inizio del progetto come nel waterfall,

messe in backlog nell'ordine in cui "devono" essere sviluppate. Non c'è nessuna prioritizzazione basata sul valore. Non c'è feedback loop con il cliente. La struttura temporale è iterativa ma la mentalità è lineare.

Pattern 4: il team fa Agile ma i manager no. Il team lavora in sprint, ma i processi aziendali intorno sono tutti waterfall: approvazioni che richiedono settimane, budget annuali fissi, richieste di pianificazione a 18 mesi, change management con lead time di sei settimane. L'Agile transformation del solo team, senza toccare la governance, produce frustrazione.

Pattern 5: si adotta SAFe perché "Scrum non scala". SAFe è un framework per organizzazioni grandi con molte squadre che devono coordinarsi. Richiede una maturità Agile di base che spesso non c'è. In mancanza di quella maturità, SAFe aggiunge complessità senza aggiungere valore. La regola pratica: prima fai funzionare un team con Scrum. Poi, quando hai dimostrato che il problema è il coordinamento tra team, valuta un framework di scaling.

La checklist del rischio prima di iniziare

Rischio | Domanda diagnostica | Segnale verde | Segnale rosso

Sponsor senza autorità | Chi ha approvato questa trasformazione? | CEO o VP con budget | Responsabile IT senza leva organizzativa

Middle management non allineato | I manager intermedi sono stati coinvolti? | Sì, dall'inizio | No, riceveranno comunicazione

PO senza autorità reale | Chi decide le priorità del backlog? | Una persona con autorità e disponibilità | Un comitato o un manager già sovraccaricato

Aspettative irrealistiche | Cosa si aspetta il management dopo 3 mesi? | Baseline + prime retrospective | Software perfetto e velocità raddoppia

Processi di rilascio lenti | Quanto ci vuole per deployare in produzione? | Ore o giorni | Settimane o mesi

Cultura del fallimento punitiva | Come vengono gestiti gli errori? | Come opportunità di apprendimento | Come prove di incompetenza

Se hai tre o più segnali rossi, la trasformazione fallirà senza affrontare prima queste cause radice.

4. Scrum in Due Pagine: Guida Operativa

I tre ruoli

Product Owner — Decide cosa costruire e in quale ordine. Gestisce il backlog, definisce le priorità, rappresenta il business e gli utenti. Ha l'ultima parola sulle priorità; il team ha l'ultima parola sul come. L'errore più comune: il PO come passacarte tra il CEO e il team, senza autorità reale.

Scrum Master — Facilita il processo e rimuove gli ostacoli. Non è il project manager, non assegna task, non controlla il lavoro. Il suo compito è rendere il team più efficace: gestisce le cerimonie, identifica impedimenti, promuove il miglioramento continuo. L'errore più comune: comportarsi da project manager centralizzando decisioni che andrebbero delegate.

Development Team — Il gruppo che costruisce il prodotto. Cross-funzionale: include tutte le competenze necessarie per consegnare un incremento funzionante. Si auto-organizza. Dimensione ideale: 3-9 persone. L'errore più comune: il team come esecutori di istruzioni, non come collaboratori nella pianificazione.

Le quattro cerimonie

Sprint Planning (2-4 ore) — Il team e il PO decidono quali storie portare nel prossimo sprint. Si definisce lo Sprint Goal: un obiettivo condiviso che dà senso allo sprint oltre alla lista di ticket. Segnale positivo: il team esce con chiarezza e confidenza. Segnale negativo: il team accetta storie senza capirle per compiacere il PO.

Daily Standup (15 minuti) — Tre domande: cosa ho fatto ieri verso lo Sprint Goal? Cosa faccio oggi? Ci sono

impedimenti? Non è un report di stato. Segnale positivo: emergono blocchi e il team collabora per risolverli. Segnale negativo: tutti riferiscono al manager, nessuno parla agli altri.

Sprint Review (1-2 ore) — Il team mostra agli stakeholder quello che ha costruito. Non è una presentazione formale — è una conversazione. Segnale positivo: il feedback cambia le priorità del prossimo sprint. Segnale negativo: è una formalità, le decisioni erano già prese.

Sprint Retrospective (1-1,5 ore) — Il team riflette su come ha lavorato. Tre domande: cosa è andato bene? Cosa può migliorare? Cosa facciamo nel prossimo sprint per migliorarlo? Si identificano 1-3 azioni concrete con responsabile e scadenza. Ogni retrospective senza almeno un'azione concreta è una retrospective fallita.

La Definition of Done

La Definition of Done è l'accordo del team su cosa significa "finito". Non "sviluppato" — finito. Senza DoD, ogni membro ha una definizione diversa di "completato". Questo crea confusione, bug in produzione e storie che riemergono sprint dopo sprint.

Template minimo:

Una storia è DONE quando:

- ☐ Il codice implementa i criteri di accettazione della storia
- ☐ Il codice è stato revisionato da almeno un altro membro del team
- ☐ I test automatizzati passano
- ☐ Non introduce regressioni evidenti sulle funzionalità esistenti
- ☐ La funzionalità è deployata e verificata in ambiente di staging
- ☐ Nessuna credenziale o secret nel codice
- ☐ Nessun dato personale in log o output di debug

Come costruirla: riunisci il team in 60 minuti, chiedi "Cosa deve essere vero affinché questa storia non ritorni nel backlog?", tieni quello che è non negoziabile. Appendila visibile a tutti. Aggiorna in retrospective.

5. DevOps per PMI: Deployare Più Veloce Senza un Team da 30 Ingegneri

Cos'è DevOps davvero

DevOps non è un ruolo. Non è la persona che gestisce i server e fa anche un po' di sviluppo. È una pratica — un modo di organizzare il lavoro che abbatte il muro tra chi sviluppa il software e chi lo gestisce in produzione.

Nel modello tradizionale, gli sviluppatori scrivono il codice e lo "lanciano oltre il muro" alle operations. Quando qualcosa va storto, inizia la fase degli accusati reciproci. Nel modello DevOps, gli sviluppatori sono responsabili del codice in produzione. Il feedback loop è rapido: un bug torna allo sviluppatore che l'ha introdotto nel giro di ore, non settimane.

Tre principi:

Shift left — Porta i controlli di qualità il più vicino possibile all'inizio del processo. Test automatizzati, linting, analisi di sicurezza — nel momento in cui il codice viene scritto, non alla fine quando è costoso correggere.

Automazione come default — Ogni processo ripetibile deve essere automatizzato. Non perché gli umani siano incapaci, ma perché gli umani fanno errori sotto pressione, e la pressione è massima quando si deploya in produzione.

Feedback loops veloci — Il tempo tra "ho scritto questo codice" e "so se funziona in produzione" deve essere il più breve possibile. Feedback lento significa correzioni tardive e costo alto.

I tre livelli di maturità DevOps per PMI

Livello 1 — Manuale Strutturato. I deploy avvengono manualmente ma esiste un processo documentato. Segnali: il deploy richiede ore di lavoro manuale, esiste una "finestra di deployment" il venerdì sera, solo una o due persone sanno come deployare. Obiettivo: riduci il rischio umano. Documenta tutto. Crea runbook per ogni operazione ricorrente.

Livello 2 — Semi-automatizzato. La build è automatizzata. I test girano su ogni commit. Il deploy in staging è automatico. Il deploy in produzione è ancora manuale ma standardizzato. Segnali: hai una pipeline CI, il deploy in staging richiede meno di 30 minuti, chiunque nel team può deployare. Obiettivo: ridurre il lead time da "codice pronto" a "codice in produzione".

Livello 3 — CI/CD Completo. Ogni commit su branch main, dopo aver superato i test automatici, viene deployato automaticamente in produzione. Il team deploya più volte al giorno. Il monitoring è proattivo. Segnali: deployment frequency più volte al giorno, lead time in minuti, MTTR sotto un'ora.

La pipeline CI/CD minima che ogni team dovrebbe avere

Una pipeline minima funzionale ha quattro fasi. Non serve di più per partire.

FASE 1: BUILD

- Compila il codice (o verifica la sintassi)
- Installa le dipendenze
- Tempo target: < 2 minuti

FASE 2: TEST AUTOMATIZZATI

- Unit test: verifica che la logica di business funzioni
- Integration test: verifica che i componenti si parlino
- Linting: verifica lo stile del codice
- Sicurezza: scan delle dipendenze vulnerabili
- Tempo target: < 10 minuti

FASE 3: BUILD ARTEFATTO

- Crea l'artefatto deployabile (Docker image, pacchetto)
- Taggalo con il numero di commit o versione
- Caricalo nel registry
- Tempo target: < 5 minuti

FASE 4: DEPLOY

- Deploy automatico in staging su ogni push al branch main
- Deploy in produzione manuale (Livello 2) o automatico (Livello 3)
- Notifica il team del risultato
- Tempo target: < 5 minuti

Tempo totale pipeline: obiettivo sotto i 20 minuti. Una pipeline che dura più di 30 minuti viene aggirata dagli sviluppatori — non aspettano il risultato, pushano e dimenticano.

****Insight 108 Vision**** — *Deployare manualmente in produzione nel 2026 non è un problema tecnico: è un problema di rischio. I dati DORA sono chiari: i team con deploy automatizzati hanno un change failure rate fino a 7 volte inferiore rispetto ai team con deploy manuali. Il motivo è semplice: il processo automatizzato è identico ogni volta. Non dimentica passaggi, non omette check sotto pressione.*

Esempio pratico: pipeline GitHub Actions

```

name: Build, Test and Deploy

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run linting
        run: npm run lint

      - name: Run tests
        run: npm test -- --coverage

      - name: Security audit
        run: npm audit --audit-level=high

  deploy-staging:
    needs: build-and-test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to staging
        run: echo "Deploying to staging..."

  deploy-production:
    needs: deploy-staging
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    environment:
      name: production
    steps:
      - name: Deploy to production
        run: echo "Deploying to production..."

```

Questa pipeline esegue build e test su ogni push, deploia automaticamente in staging, e richiede approvazione manuale per la produzione. È un punto di partenza — puoi aggiungere notifiche Slack, report di coverage, scan di sicurezza più approfonditi.

6. I 5 Quick Win che Puoi Iniziare Questa Settimana

Questi cinque interventi non richiedono budget, non richiedono approvazioni, non richiedono una trasformazione organizzativa. Ognuno produce un risultato misurabile entro 30 giorni.

Quick Win 1: La Retrospective di 30 Minuti.

La retrospective e il meccanismo di miglioramento continuo più potente per un team. L'unico momento in cui il team si ferma, guarda come sta lavorando, e decide di cambiare qualcosa.

Struttura base: 5 minuti di check-in (una parola su come ti senti), 10 minuti su cosa è andato bene, 10 minuti su cosa può migliorare, 5 minuti per identificare 1-2 azioni concrete con responsabile e scadenza. Regola fondamentale: ogni retrospective deve produrre almeno un'azione concreta. Una retrospective che produce solo liste di problemi e una retrospective fallita.

Quick Win 2: Il Backlog Visibile a Tutto il Team.

La trasparenza sulle priorità riduce il "lavoro sommerso": attività non tracciate, richieste informali, side project che consumano capacità senza comparire nelle pianificazioni. Regola critica: se un'attività non è sulla board, non esiste. Qualsiasi richiesta — dal CEO, dal cliente, dall'account — deve passare per il backlog. Nessuna eccezione. Questa regola da sola riduce significativamente il caos operativo.

Quick Win 3: La Definition of Done Condivisa.

Sessione di 60 minuti con il team. Risultato atteso entro 30 giorni: riduzione delle storie che "tornano indietro" e dei bug post-deploy che avrebbero potuto essere catturati prima.

Quick Win 4: Un Deploy Automatizzato, Anche se Piccolo.

Non serve automatizzare tutto subito. Inizia con il passo più piccolo possibile. Se non hai nessuna automazione, inizia con i test automatizzati prima del deploy. Anche solo "i test devono passare prima di deployare" è un miglioramento rispetto al nulla. Se il deploy è completamente manuale, crea uno script che esegue tutti i passi in sequenza — anche se lo esegui ancora manualmente, elimina gli errori di memoria.

Quick Win 5: Le Metriche DORA su Carta.

Crea una tabella con queste colonne: data deploy, versione, ambiente, durata deploy, problemi, MTTR. Dopo 4-6 settimane hai la tua baseline. Non puoi migliorare quello che non misuri.

7. Come Misurare se Sta Funzionando: Le 4 Metriche DORA

Le metriche DORA (DevOps Research and Assessment) sono quattro numeri che predicono meglio di qualsiasi altra metrica la qualità del processo di delivery.

Deployment Frequency — Quanto spesso il team deploia in produzione? Per una PMI italiana, l'obiettivo realistico nel primo anno è passare da "una volta al mese" a "una o più volte alla settimana". Deploy frequenti significano modifiche piccole, rischio basso, errori facili da isolare.

Lead Time for Changes — Quanto tempo passa dal commit del codice al deploy in produzione? Se scopri un bug critico e ci vuole una settimana per deployare la correzione, hai un problema di lead time. Se ci vogliono 20 minuti, hai un vantaggio competitivo concreto.

Change Failure Rate — Che percentuale dei deploy causa problemi in produzione? Un CFR alto non significa che il team è incompetente — spesso significa che i test automatizzati sono insufficienti o che l'ambiente di staging non è rappresentativo della produzione. Target: sotto il 15%.

Mean Time to Recovery — Quando c'è un problema in produzione, quanto ci metti in media a risolverlo? Un MTTR basso richiede monitoring che rileva il problema rapidamente, processo di rollback ben definito e testato, team con le competenze per diagnosticare velocemente. Non puoi evitare tutti i problemi — ma puoi decidere quanto velocemente li risolvi.

Benchmark realistici per PMI:

Metrica	Punto di partenza tipico	Obiettivo 6 mesi	Obiettivo 12 mesi
Deployment Frequency	1 volta al mese o meno	1-2 volte a settimana	Piu volte a settimana
Lead Time	2-8 settimane	1-5 giorni	Meno di 1 giorno
Change Failure Rate	40-60%	Sotto il 30%	Sotto il 20%
MTTR	Giorni o settimane	Meno di 1 giorno	Meno di 4 ore

I segnali qualitativi che la trasformazione sta funzionando

Le metriche DORA sono necessarie ma non sufficienti. Questi segnali indicano che il cambiamento sta avvenendo a livello culturale:

Segnali positivi: il team solleva i problemi in retrospective prima che diventino crisi; gli sviluppatori sono entusiasti di mostrare il loro lavoro nella Sprint Review; i deployment avvengono senza stress — sono eventi ordinari, non emergenze; il team parla del "perche" di una storia, non solo del "cosa"; si fanno domande come "ha senso fare questo adesso?" invece di eseguire ciecamente.

Segnali di allarme: le retrospective producono sempre le stesse azioni senza mai risolvere nulla; il team evita di sollevare problemi perche "tanto non cambia niente"; la velocity viene usata come obiettivo invece che come strumento di pianificazione; le storie vengono stimate sistematicamente in modo ottimistico per compiacere il management.

***Insight 108 Vision** — Il "silenzio nella retrospective" e uno dei segnali piu preoccupanti. Se nessuno porta problemi, non e che tutto va bene: e quasi sempre che le persone non si sentono sicure di parlare. Il passo successivo non e un tool migliore — e costruire un ambiente in cui gli errori siano informazioni, non condanne.*

8. Il Piano d'Azione 30/60/90 Giorni

Questo piano presuppone un team di 3-15 persone, esperienza Agile minima o nulla, e un punto di riferimento — leader tecnico, engineering manager, o consulente — che guidi il processo.

Giorni 1-30: Fondamenta

Checklist operativa:

- ☐ Intervista ogni membro del team (30 minuti ciascuno): quali sono i pain point reali?
- ☐ Misura le metriche DORA baseline su un foglio di calcolo
- ☐ Identifica il Product Owner reale: una persona con nome e autorita
- ☐ Crea e priorizza il backlog iniziale (20-30 storie)
- ☐ Costruisci la Definition of Done in sessione condivisa di 60 minuti
- ☐ Scegli la board (Trello, GitHub Projects, Jira, post-it fisici — va bene qualsiasi)
- ☐ Definisci il ritmo delle cerimonie con un calendario ricorrente
- ☐ Fai il primo Sprint Planning tenendo le aspettative basse: e un esperimento
- ☐ Conduci la prima retrospective con almeno 1 azione concreta implementata

Giorni 31-60: Automazione e Ritmo

Checklist operativa:

- ☐ Fai l'audit del processo di deploy attuale: documenta come deployate adesso
- ☐ Scegli la piattaforma CI/CD (GitHub Actions, GitLab CI)
- ☐ Implementa la pipeline fase 1: build + test automatici su ogni push

- [] Testa la pipeline su un branch non critico prima di applicarla al main
- [] Conduci tre retrospective con azioni implementate
- [] Verifica che il backlog sia regolarmente curato dal PO
- [] Il team conosce le proprie metriche DORA e capisce i numeri

Giorni 61-90: Ownership e Ottimizzazione

Checklist operativa:

- [] Aggiungi scan di sicurezza alla pipeline (dipendenze, secret)
- [] Implementa alerting base su produzione (uptime, error rate)
- [] Testa e documenta il processo di rollback
- [] Misura il lead time end-to-end con precisione
- [] Rivedi e aggiorna la Definition of Done basandoti sull'esperienza dei due mesi precedenti
- [] Confronta le metriche DORA post-trasformazione con la baseline
- [] Il team organizza le cerimonie senza bisogno di essere ricordato
- [] Il backlog è pulito: meno di 50 storie aperte, tutte priorizzate

Tabella riassuntiva:

Periodo	Focus	Obiettivo misurabile
0-30 giorni	Fondamenta	Baseline DORA + DoD + prime cerimonie
31-60 giorni	Automazione e ritmo	Pipeline CI/CD + 3 retrospective con azioni
61-90 giorni	Ownership e ottimizzazione	Team autonomo + metriche DORA migliorate

9. Glossario Rapido

Termine	Significato
Agile	Approccio allo sviluppo basato su consegne frequenti, feedback continuo, adattamento al cambiamento
Scrum	Framework Agile con sprint, ruoli definiti e cerimonie cadenzate
Kanban	Metodo visuale per gestire il flusso di lavoro continuo, senza sprint fissi
Sprint	Iterazione temporale fissa (solitamente 2 settimane) verso uno Sprint Goal
Definition of Done	Accordo condiviso su cosa significa che un'attività è davvero completata
Velocity	Quantità di lavoro completata in uno sprint. Strumento di pianificazione, non obiettivo
DevOps	Pratica che abbate il confine tra sviluppo e operations per accelerare il delivery
CI/CD	Continuous Integration / Continuous Delivery — pipeline automatizzata di build, test, deploy
Lead Time	Tempo dal commit del codice al deploy in produzione
DORA	DevOps Research and Assessment — le 4 metriche chiave del delivery
Change Failure Rate	Percentuale di deploy che causano problemi in produzione
MTTR	Mean Time to Recovery — tempo medio per risolvere un problema in produzione
ADR	Architecture Decision Record — documento che traccia una decisione architetturale

10. Risorse per Andare Piu in Profondo

Libri con il miglior rapporto tra tempo investito e impatto pratico:

Accelerate — Nicole Forsgren, Jez Humble, Gene Kim. Il libro che ha reso famose le metriche DORA. Basato su ricerca empirica su migliaia di team. Se leggi un solo libro su DevOps, leggi questo.

The Phoenix Project — Gene Kim, Kevin Behr, George Spafford. Romanzo di business che spiega DevOps attraverso

una storia. Utile per convincere i non tecnici.

Team Topologies — Matthew Skelton, Manuel Pais. Come organizzare i team per ridurre il cognitive load e migliorare il flusso di lavoro.

Continuous Delivery — Jez Humble, David Farley. Il riferimento tecnico su pipeline CI/CD e deployment automation.

Empowered — Marty Cagan, Chris Jones. Come costruire team di prodotto con ownership reale. Per chi vuole andare oltre Scrum.

Vuoi Andare Oltre?

Vuoi applicare questo metodo alla tua azienda? Prenota 30 minuti con noi su 108vision.it — gratuito, senza impegno.

In quella conversazione: capiamo dove sei oggi, identifichiamo i 2-3 interventi ad alto impatto per il tuo contesto specifico, e ti diamo una valutazione onesta di cosa puoi fare in autonomia con questo manuale e dove un supporto esterno accelererebbe i risultati.

108 Vision — Costruiamo la direzione, non solo il codice.